



Hand gesture recognition-based non-touch character writing system on a virtual keyboard

Md. Abdur Rahim¹  • Jungpil Shin¹  • Md. Rashedul Islam¹ 

Received: 1 March 2019 / Revised: 5 September 2019 / Accepted: 7 November 2019 /

Published online: 8 January 2020

© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract

The non-touch system is a modern approach of computer-interface technology capable of revolutionizing human-computer interaction. The interface allows the user to input data and interact with a human, machine or robot in an uncontrolled environment, treatment or industrial life. However, it is challenging to input data into the machine and interact with man and machine with a variety of complexities such as cluttered environment, gesture tracking, and speed. There are many evolving systems, for example, aerial handwriting, sign language recognition, and finger alphabet recognition require substantial effort for all character learning and overhead processing, thence the accuracy of the classification is reduced. Therefore, this paper proposes a non-touch character writing system that allows users to interact and manage the on-screen virtual keyboards in a secure and healthy way by recognizing few hand gestures. We divide this work into two parts: a) hand gesture recognition; and b) gestural flick input using a virtual keyboard. A user-friendly keyboard interface is displayed on the monitor, which uses a flick input method. A deep learning method with CNN is used to extract the features of a gesture. To determine these features, color segmentation is used to detect the hand; color pixels can be obtained by extracting a particular HSV (hue, saturation, value) and applying threshold masking to the input image. Finally, a support vector machine is used to give a more accurate classification of the hand gestures. The user uses a gestural flick input system to perform non-touch character input and enters the character by viewing the virtual keyboard. The character input is executed based on the recognition of the user's hand gestures. Character input is evaluated based on the average classification accuracy of hand gestures and character recognition, and the accuracy and speed of input. Then, the system is compared with the state-of-the-art algorithms. The experimental results show that the proposed system can recognize seven typical gesture functions and input characters with 97.93% accuracy, which demonstrate the superiority compared to the state-of-the-art algorithms.

Keywords Human-computer interaction (HCI) · Gestural flick input · Virtual keyboard · Character input · Convolutional neural network (CNN)

✉ Jungpil Shin
jpshin@u-aizu.ac.jp

1 Introduction

A non-touch system is an advanced computer interface technology used to enhance human-computer interaction (HCI) [1]. The interface helps to interact with a computer, machine or a robot in case of an unavoidable situation, security and treatment or industrial life. For the advancement of sensors and cameras, the non-touch devices and interfaces will continue to integrate into daily life. In recent years, HCI has improved the performance of applications involving gesture recognition. In particular, hand gesture recognition plays an important role as a technique for the development of HCI, since it provides a natural and direct way for users to express their feelings [2–5], interact with virtual reality (VR) [6] or augmented reality (AR) [7] devices, human-machine interaction [8], and to play games [9, 10]. Many prior studies have also focused on the recognition of device-based gestures. This type of processing requires specialized hardware and dedicated devices [11, 12], or wearable devices [13], and these technologies are inconvenient and difficult to use in real life since they require users to be in direct contact with additional devices. However, both touch and non-touch devices have become widespread in various types of accessible devices, and especially in tablets, PCs and smartphones. A keyboard and mouse are usually used to input characters to computers, and software keyboards to input characters to smartphones or tablets. However, users may feel unsafe using these touch-based devices for sensitive applications, as hackers can collect user data and even biometrics, which may cause security problems. In non-touch techniques, however, the user does not need to type on a keyboard, write on a trackpad/touch screen, or control a smart system in many applications involving text applications. There is also a huge demand in the industry to provide advanced and comprehensive services to ensure a healthy and safe environment, including avoiding risky situations. In order to meet the highest demand, many industry applications have used advanced human-machine interface (HMI) technology [14]. Inspired by this technology, gesture recognition has been customized for a non-touch character input system and this has improved the progress of such applications. Shin and Kim proposed a non-touch character input method based on hand-tapping gestures to input English and Japanese characters, which can be used to simplify HCI [15]. Hand-tapping gestures can be effectively used as an alphabet, and this is accomplished using an aerial virtual keypad. However, it is difficult for the user to remember the input characters for the hand tapping gestures, and it is also difficult to detect the fingertip when the input of the hand tapping gestures overlaps the fingers. Consequently, in this paper, we propose a non-touch character input system that incorporates hand gesture recognition, where seven different gesture functions (i.e. newline, space, delete, backspace, open, close, and select language) can be recognized with a high level of accuracy in a real working environment. To achieve these goals, we need to solve two major underlying challenges:

- 1) Hand gesture recognition gives rise to numerous errors in an uncontrolled environment, since it relies on real-time processing, and thus requires an effective and practical technique for implementing these interfaces.
- 2) Hand-gesture-based non-touch interfaces and writing techniques are required in order to provide a more intuitive HCI with a virtual keyboard and gestural flick input system.

The points of focus and research contributions of this study are as follows:

- 1) A non-touch interface based hand gesture recognition system is introduced in an uncontrolled environment. We apply threshold masking to deal with the use of uncontrolled

environments and variations in lighting when segmenting the hand. Threshold values are imposed according to various tests, and show outstanding performance in this regard. We therefore use a deep learning technique involving a CNN to extract the features and train the model activity in the system, and we conduct an extensive evaluation using a support vector machine (SVM) to validate our design.

- 2) A non-touch writing system can be used when the environment is not ideal for contact with people or devices or for communicating verbally. This study investigates the suitability of a feature extraction method for various properties in order to solve the problem of the recognition of different hand gestures.
- 3) We present a virtual keyboard based on character and non-character blocks, and use a flick input method for this keyboard in which the user performs the character input through particular hand gestures.

This paper is organized as follows. In Section 2, we review related work. We present a description of the technical details for our proposed system in Section 3. In Section 4, we evaluate our system and report the experimental results. In section 5, we briefly explore the experimental results and finally conclude the paper in section 6.

2 Related work

Today, non-touch devices and interfaces are becoming more diversified and are undergoing continued development. Traditionally, a keyboard and mouse have been used as input devices for computers; however, researchers are developing other forms of interaction via HCI where speed and hand tracking are important for interactive applications. Gesture recognition [16] provides a simple and intuitive approach to these interactions. However, the recognition of different shapes and directions of gestures even for the same person, and the use of image processing techniques that are complex and require overhead processing. Many researchers aim to use simple gestures for computer control and to provide a system for inputting characters [17] or recognizing letters/words [18, 19]. It consists of gesture modeling, analysis and recognition using machine and deep learning techniques, and is often applied to groups with specific needs. However, overlapping finger attempts may concern the effectiveness of gesture recognition. Besides, the author proposed glove based tracking to track the hand [20]. However, it is uncomfortable for long time use. In [21], authors proposed an eye-writing inputs system for HCI. These methods provide an alternative direction for the input system, and more accurate research is needed for recognition. A system of continuous recognition of text written in the air based on accelerometers and gyroscope sensors that serve as an input device for wearable computers [22]. However, there is a noise associated with data that makes it difficult to process incoming data classifications, also it is inconvenient to connect a device at all times.

Li et al. [23] presented a system that enables lecturers to write and control fluently on a non-touch screen via visible in-air gestures. However, these systems are limited by the complexity of the tablet system. There are many methods that have been used to determine the location of the palm, such as finger detection, joint location or identification of anchor points [24, 25]. In [26], the author proposed a distance metric to handle noise from the Kinect sensor, which measures the inequality in hand shapes. However, the shape recognition method is not robust to significant distortion of the hand shapes. Murata and Shin proposed a DP-matching-based numeric and alphabet character recognition system [27]. The image processing in this problem

is very complex, and a huge computational time is required. Deep learning methods have been used to solve problems in various fields of gesture recognition in recent years. In [28], Neverova et al. used a convolutional neural network (CNN) to recognize the Italian language by combining RGBD data from the hand area with skeletal motion. Their strategy was designed to perform interior gestures, and the identification of skeletal position in an uncontrolled environment is very complex. Oyedotun et al. [29] proposed a deep-learning-based recognition method that could differentiate hand gestures from Thomas Moeslund's gesture detection database, and achieved recognition of 24 American Sign Language gestures. Recently, deep learning methods have achieved significant results in various computer vision activities, and the use of deep learning is attracting attention in terms of gesture recognition [30–32]. A novel recognition algorithm is presented in [33]. The author performed image preprocessing, de-noising and edge detection of gesture input images to give hand edge images; these hand gesture and edge images were used as two input channels for the CNN. As well as allowing for quick responses to the user and high accuracy in gesture recognition, a good sustaining process is required for fast response in this application.

3 Proposed model

This section presents the proposed model and the basic workflow of our system. The input information is acquired via a webcam and processed using the proposed model. The workflow is divided into three segments: a) a virtual keyboard; b) a hand gesture recognition system; and b) a gestural flick input. The virtual keyboard is displayed on the monitor, and allows the user to input characters and to perform different gestural functions. The hand gesture recognition system detects and recognizes the area of the hand that is executing the different gesture functions, such as “newline”, “space”, “delete”, “backspace”, “open”, “close”, and “select language”. In the gestural flick input system, the user inputs the character without touching an input device. Figure 1 shows the general workflow of the proposed system.

3.1 Virtual keyboard

The system displays the virtual keyboard on the monitor. The virtual keyboard uses the flick input method, a Japanese input method used on smartphones that is now also used in other countries. Flick input is often used as a way to input characters on a smartphone, as it allows users to easily enter characters. In this system, users can choose a particular character instead of pressing a key repeatedly. Figure 2 represents a flick keyboard for the input of Japanese and English characters. In this work, the virtual keyboard contains two processes, a primary virtual keyboard (PVK) and a secondary virtual keyboard (SVK). The system displays the PVK on the monitor. The user inputs a character by using the PVK, as shown in the example in Fig. 3a. The PVK is divided into 13 blocks. The first 12 are characters of the flick character sequence, and are known as character blocks; the remaining one is known as a non-character block, and is used for the performance of different gesture functions. When the user performs gesture functions in character blocks via the PVK, the system displays the SVK to offer additional activity. The SVK presents a flick character sequence, such as a, i, u, e, and o, as shown in Fig. 3d. Five characters are used in each flick input sequence shown in Fig. 3e. Each character block in the PVK and SVK is expressed as a region of interest (ROI) location in order to capture the hand gesture images. This system supports a virtual keyboard offering three character sets: Japanese Hiragana, the English alphabet and numbers as

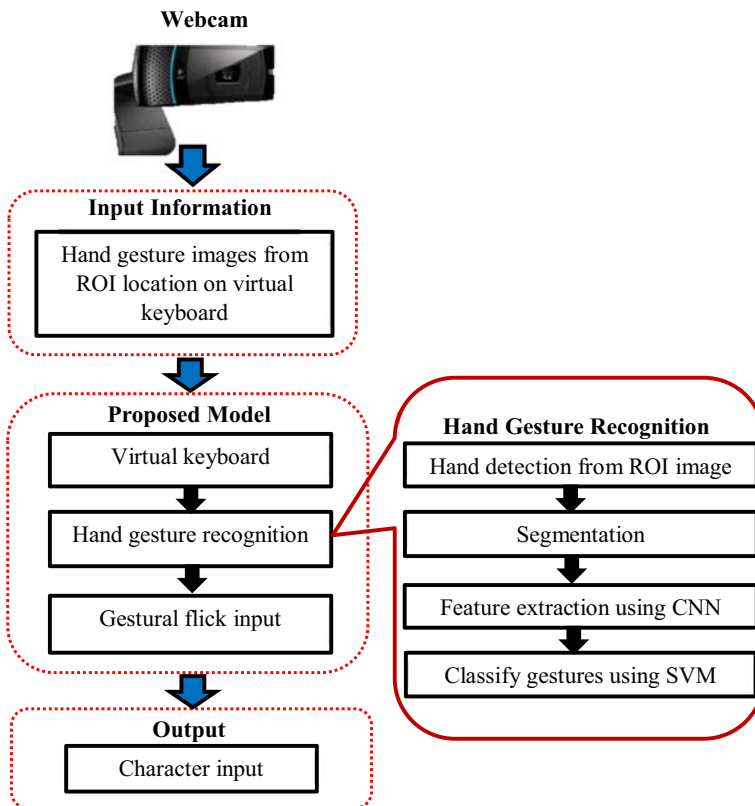


Fig. 1 General workflow of the proposed system

shown in Fig. 3a–c, respectively. Certain special characters and operators are also included in the English and numerical virtual keyboards. Table 1 presents the flick character sequences of the virtual keyboard. Finally, the user performs the character input by following this flick character sequence. For example, if a user wants to enter a character ‘f’, he or she moves to English PVK. Then select the first character in the sequence of the corresponding character, that is ‘e’ and perform the gesture functions. The system displays the SVK as in Fig. 3e. The user then selects the appropriate character ‘f’ and performs a gesture function in the corresponding block to input the character.

3.2 Hand gesture recognition

A hand gesture recognition (HGR) system is classified into several basic steps involving acquisition of the image, detection and segmentation of the hand, hand tracking and finally the classification of gestures [34]. However, since our recognition process can work at the same rate as acquisition of the image, tracking is not required. Thus, our HGR system consists of image acquisition, segmentation and identification, extraction of features, and gesture recognition. Section 3.2.1 briefly describes the process of hand detection and gesture image segmentation, and Sections 3.2.2 and 3.2.3 describe the process of feature extraction and the details of the classification of gesture functions.

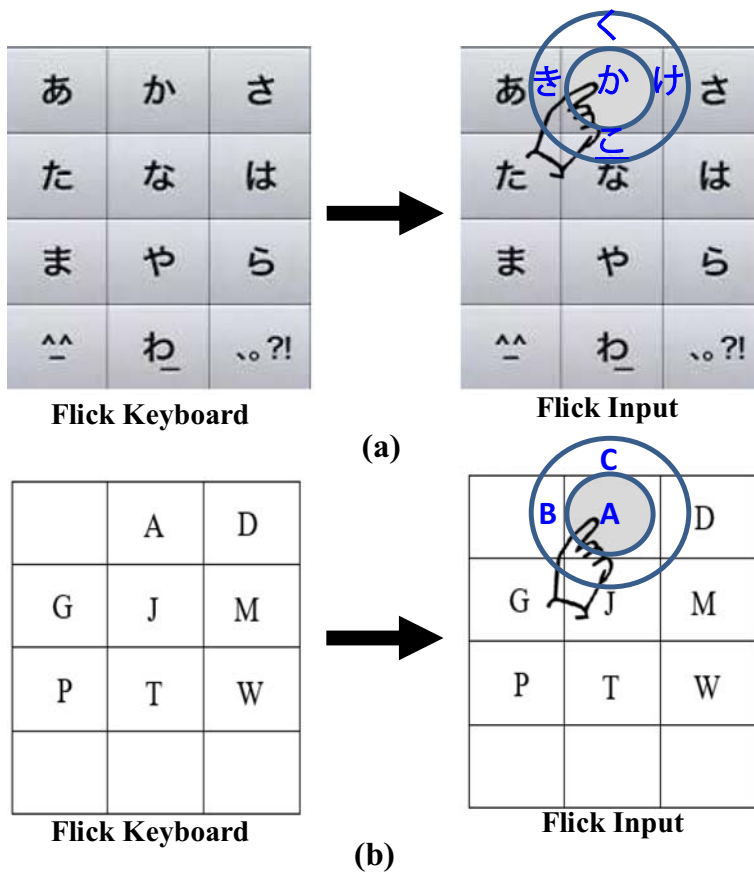


Fig. 2 A flick keyboard used in smartphones: **a** Japanese, **b** English

3.2.1 Hand area detection from ROI images using segmentation

The system captures user gestures via an interface with a webcam. The input to the system is an image frame of a static hand gesture from live video. To resolve the unwanted area of the video frame to detect the hand gesture, a specific area(s) of the virtual keyboard is imposed as the ROI containing only a hand gesture image. In order to increase our ability to detect the

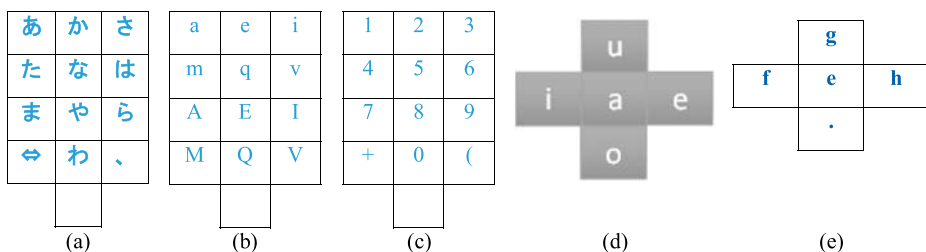


Fig. 3 Proposed primary virtual keyboard **a** Japanese, **b** English, and **c** Numeric; **d** flick input; **e** secondary virtual keyboard

Table 1 Flick character sequences of the virtual keyboard

Virtual keyboard language					
Japanese (Hiragana)		English		Number	
First character	Flick character sequence	First character	Flick character sequence	First character	Flick character sequence
あ	あ, い, う, え, お	a	a, b, c, d, _	1	1, , , ,
か	か, き, く, け, こ	e	e, f, g, h, .	2	2, , , ,
さ	さ, し, す, せ, そ	i	i, j, k, l, ,	3	3, , , ,
た	た, ち, つ, て, と	m	m, n, o, p, ;	4	4, , , ,
な	な, に, ぬ, ね, の	q	q, r, s, t, u	5	5, , , ,
は	は, ひ, ふ, へ, ほ	v	v, w, x, y, z	6	6, , , ,
ま	ま, み, む, め, も	A	A, B, C, D, @	7	7, , , ,
や	や, ,ゆ, ,よ	E	E, F, G, H, #	8	8, <, >, /, #
ら	ら, り, る, れ, ろ	I	I, J, K, L, :	9	9, !, ?, &,
⇒	⇒, ", , 小,	M	M, N, O, P, '	+	+, -, *, /, ^
わ	わ, ゐ, ゑ, ,を	Q	Q, R, S, T, U	0	0, [,], ~, \$
、	、, ,。 , ?, !,	V	V, W, X, Y, Z	(	(,), {, }, =

hand, we identify those pixels that match the color of the hand in a particular frame and perform color segmentation. In this segmentation process, we convert the RGB color space and use HSV color model analysis by thresholding the saturation (S) and value (V) elements. In HSV representation, the hue, saturation, and values determine the color of the hand, the intensity of the color and the lightness of the image, respectively. A Gaussian blurring technique is used in the input to smooth the image and reduce the noise. We also use threshold masking for image segmentation and apply multiple masks for color segmentation. Low and high threshold masks are used for hue, saturation, and value in which the pixels are set to one, and the remaining pixels to zero. We use different thresholds for different areas of the same image, so that it is well segmented in the hand area under different forms of illumination. Threshold values are fixed according to various tests depending on the training dataset. The minimum (S_{low}) and maximum (S_{high}) limits of saturation are 0.01 and 1, respectively, while the values of V_{low} and V_{high} are 0 and 1, respectively. When the threshold is not applicable, the hue components of h_{low} and h_{high} are 0 and 1, respectively. The system identifies the color of the hand if Eq. (1) is satisfied:

$$P_{gesture}(i, j) = \begin{cases} 1, & \text{if } h_{low} < H(i, j) < h_{high}, \quad S_{low} < S(i, j) < S_{high}, \quad v_{low} < V(i, j) < v_{high} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

where $H(i, j)$ is a hue element, $S(i, j)$ is a saturation element, and $V(i, j)$ is a value element in the spatial domain. Figure 4 shows the color analysis of a hand gesture image, in which the output is a processed image that acts as input for determining the features.

3.2.2 Feature extraction using CNN

A CNN is a deep learning model with multiple layers that are responsible for converting input into a generic representation [35]. A CNN can be used to conduct feature extraction, since multilayered features can share common features. CNN involves different networks that can extract potential features for classified models. AlexNet is among the most popular networks for this functionality,

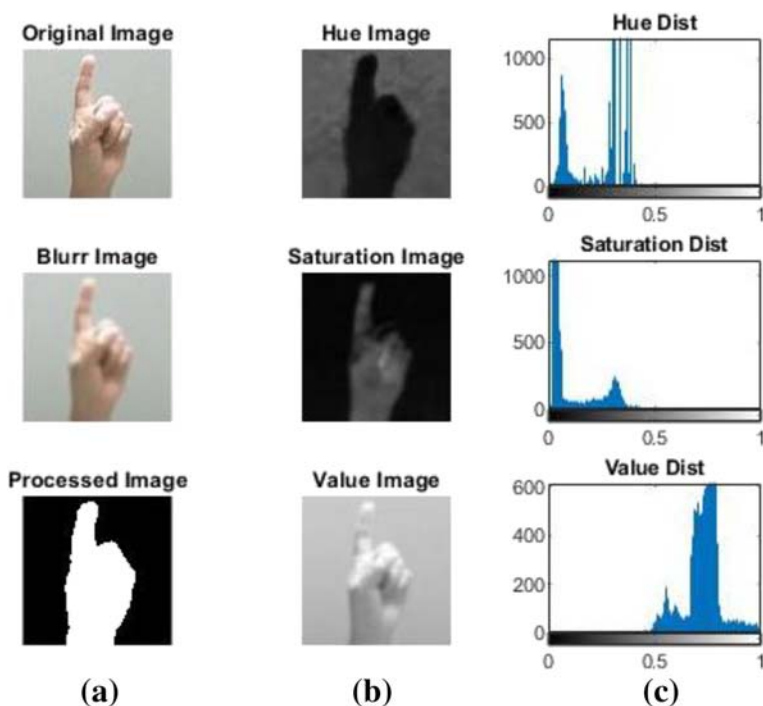


Fig. 4 Color analysis of a hand gesture image: **a** input, blurred and processed image (top to bottom, respectively); **b**, **c** HSV distribution of the color of the hand and histogram of the input image (top to bottom, respectively)

and includes five convolution levels and three fully connected network layers [36]. AlexNet measures the simplicity and expertise in a shallow model combined with a deep model prediction of performance. It has numerous filters per layer, and deep with stacked convolutional level. The convolutional level is to detect the input local features and move the convolution through the fixed length of the kernel steps. Since the convolution kernel is fixed, but the weight parameters are obtained through the running layer by layer. Figure 5 shows the architecture of the feature extraction model. The input levels are defined in the first layer, and the size of the input image is $128 \times 128 \times 3$. The first convolutional layer filters the input images with a 4-pixel stride with 96 kernels of $11 \times 11 \times 3$ size. The output of the first convolutional layer acts as the input to the second convolutional layer and filters it with a 256 kernel of $5 \times 5 \times 256$ size. The others convolutional

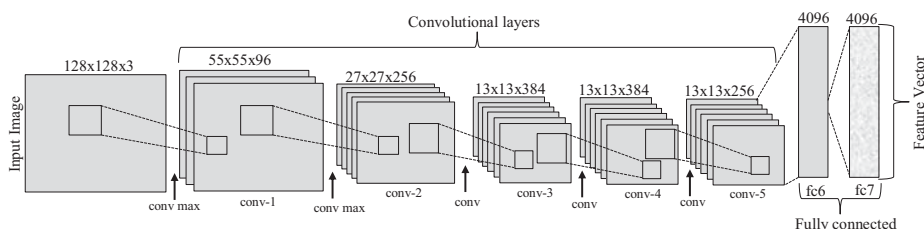


Fig. 5 The CNN model for feature extraction

layers (third, fourth and fifth) are connected to each other with no interference in the pooling or normalization. The 384 kernels of size $3 \times 3 \times 256$ of third convolutional layer is connected to the outputs of the second convolutional layer. The fourth and fifth convolutional layers has 384 and 256 kernels of size $3 \times 3 \times 192$ and $3 \times 3 \times 192$, respectively. There are 4096 neurons in each fully connected layer. This increases the size of our training as determined by a factor of 4096 although the training examples of the results are certainly highly interdependent. The main objective of the fully connected layer is to use the extracted features to classify input images into different gesture functions, based on the training dataset. Here, we use processed images as input and use these to extract the features. The deep convolutional network automatically learns the characteristics of a hierarchical image. In the lower layer, low-level features (for instance corners and edges) are extracted; the mid-level features (for instance colors and shapes) are extracted in the middle layer; and an image object is represented in the higher layers. The convolutional layer extracts the features and the fully connected layers are responsible for recognition. However, only a few layers are required to extract image features that are suitable for a CNN. The ‘fc7’ layer is used to extract the features in this paper. The initial image features are obtained via the use of this layer at the beginning of the network.

3.2.3 Classification using SVM

We use an SVM classification algorithm to deal with hand gesture classification and recognition. SVM is used to classify data by creating a hyperplane or set of hyperplanes in a high-dimensional space [37]. Although SVM is essentially a two-class classifier, it can be expanded into a multiclass classifier, and can use one-versus-many (OVM) and one-versus-one (OVO) discriminatory strategies. An effective SVM method is to use a hyperplane to group input data into two separate sections. In many cases, datasets are not linear, and are unable to be classified into two sections in the hyperplane. In our method, we therefore invoke the kernel function, which solves this problem by classifying nonlinear datasets. The radial basis function (RBF) was selected from the range of possible types of nonlinear kernel function (for instance Gaussian radial, polynomial, hyperbolic tangent functions etc.) as shown in Eq. (2):

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right), \sigma > 0 \quad (2)$$

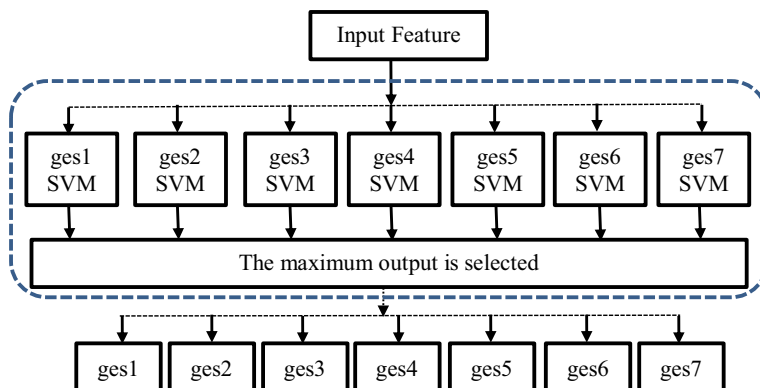


Fig. 6 OVM-MCSVM structure for identifying gesture functions

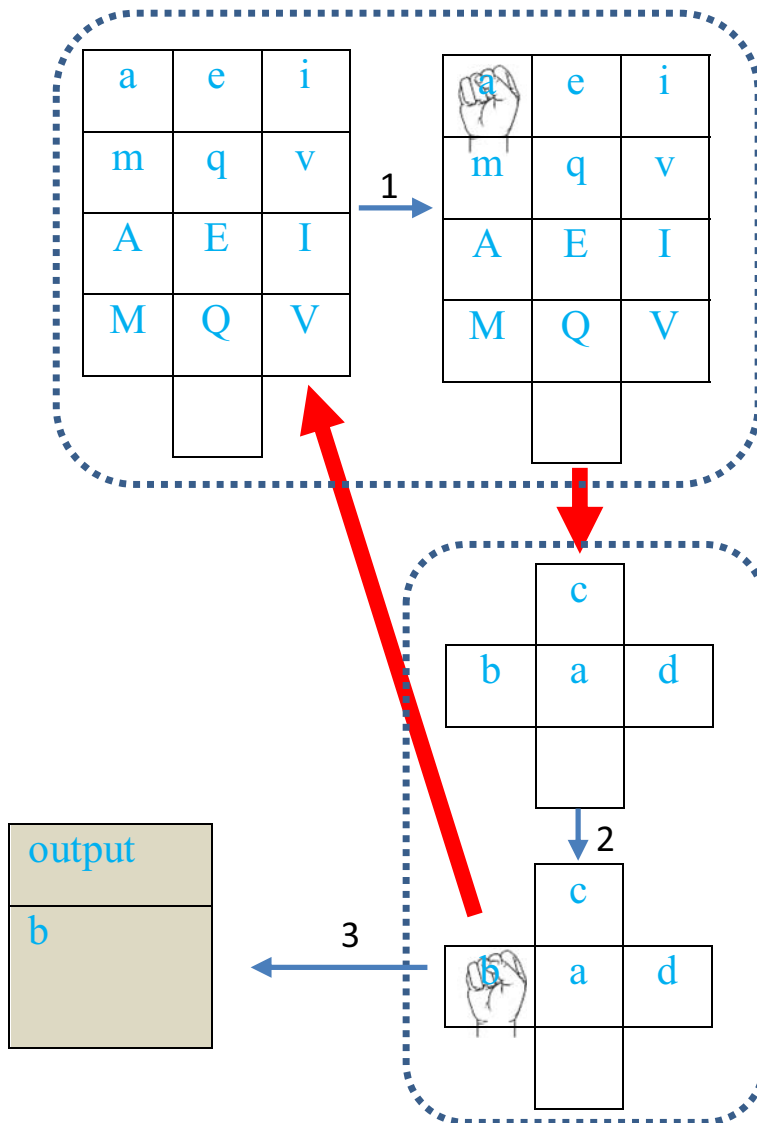


Fig. 7 The gestural flick input system

Table 2 Description of different gesture functions

Gesture function	Description
Newline	Recognized by index finger
Space	Recognized by two fingers (index and middle finger)
Delete	Recognized by three fingers (index, middle and ring finger)
Backspace	Recognized by four fingers without thumb
Open	Recognized by five fingers
Close	Recognized by the hand without showing fingers
Select language	Recognized by the thumb



Fig. 8 Examples of the hand gesture dataset images

where the two different input parameters of the processing function of k are x_i and x_j , and σ is an independent parameter that is required to find the effective basis kernel function. In the experiments we used $\sigma = 0.7$ as the basis for trial and error. The RBF function is straightforward, it can achieve nonlinear mapping, and uses relatively few parameters. In this work, there are seven (7) SVMs for seven (7) gesture classes. Due to the complexity of the multi-class non-linearly classified system, OVM is used in our proposed system. The OVM-MCSVM consists of seven SVMs working in parallel, as shown in Fig. 6. In each SVM, one class varies from another class and the result is calculated by obtaining the maximum output value of the SVM.

3.3 Gestural Flick input

The gestural flick input is a non-touch character input method by which the user inputs characters through a virtual keyboard on the display screen. It is executed via recognition of the user's hand gestures. In essence, the user can select and input a character from the character block of the virtual keyboard, using 'open' and 'close' gestures, and can perform different gesture functions via non-character blocks. The gestural flick input system is shown in Fig. 7, and involves the following procedures:

1. The user selects the first character of the flick character sequence by viewing the PVK and performing the 'close' gesture on that character block. The system displays SVK on the monitor.
2. The user selects the appropriate character by viewing the SVK and performing the 'close' gesture on that character block.
3. The selected character is input, and the system displays the PVK on the monitor. The user inputs characters by repeating these procedures.

The user also performs various hand gestures as gesture functions in the character input system. The user executes the gesture functions via the non-character block; these gesture functions are described in Table 2. The user is able to enter a new line by holding up the index finger, and can add a space by holding up two fingers (index and middle finger). When three fingers (index, middle and ring finger) are held up, the system deletes a previously input character. The backspace is executed when four fingers are displayed. The system detects an



Fig. 9 Examples of the hand gesture processed images

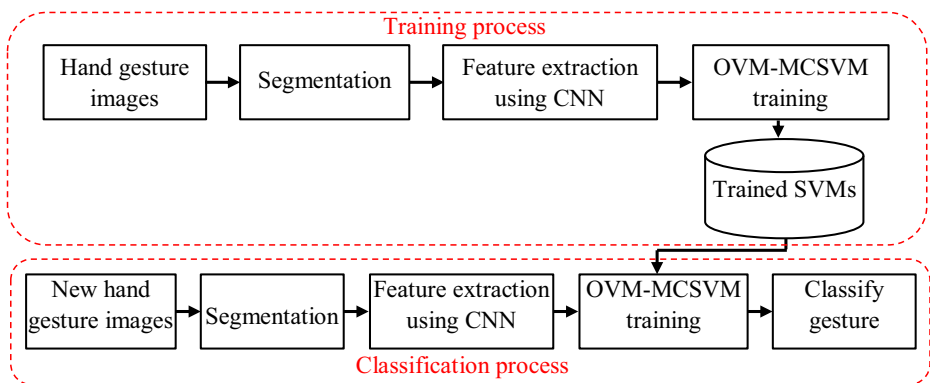


Fig. 10 Procedure of the hand gesture recognition method

‘open’ gesture when all the fingers of the right hand are displayed, and a ‘close’ gesture when no fingers are displayed. The ‘open’ and ‘close’ gestures are used for entering characters. An ‘open’ gesture does not perform any gesture activity to input characters or other gesture functions. The thumb is used to select the language of the virtual keyboard.

4 Experimental results and evaluation

In this section, we conduct experimental testing in a real environment and assess the performance of our proposed system in terms of recognition accuracy and robustness. We conduct extensive evaluations of our dataset for gestural performance and real-time evaluation with non-touch character input using the virtual keyboard.

4.1 Experimental setting and datasets processing

An experimental setup was established in the laboratory. For the hand gesture images, a webcam was connected to a computer (Intel Core i5–2400, 3.10 GHz, GeForce GTX 1080 Ti) and mounted at an appropriate place. The user was positioned at the appropriate distance from

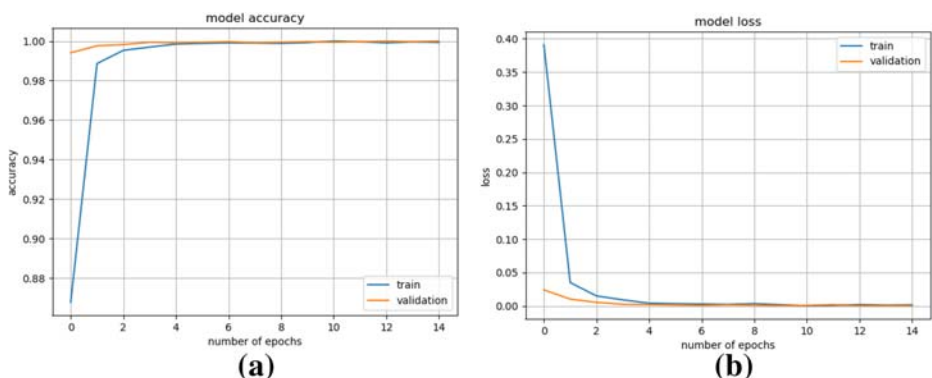


Fig. 11 Hand gesture recognition model: **a** model accuracy, and **b** model losses

Table 3 Average recognition accuracy of different gesture functions for each subject

Subject	Gesture							
	Newline	Space	Delete	Backspace	Open	Close	Select Language	Avg. of avg. accuracy
Subject 1	98.33	96.67	95	95	100	100	96.67	97.38
Subject 2	96.67	100	96.67	96.67	100	100	98.33	98.33
Subject 3	100	96.67	98.33	98.33	100	100	100	99.05
Subject 4	100	98.33	95	96.67	100	100	98.33	98.33
Subject 5	96.67	96.67	96.67	95	100	100	96.67	97.38
Subject 6	98.33	98.33	98.33	93.33	100	100	98.33	98.10
Avg. of avg. accuracy	98.33	97.78	96.67	95.83	100	100	98.06	98.09

the webcam so that the area of the hand is placed in the ROI location of the virtual keyboard, and entered characters as input.

The hand gesture dataset contains the images of seven (7) typical hand gestures (newline, space, delete, backspace, open, close, and select language), and these were performed by six (6) subjects. Figure 8 shows examples of typical hand gesture images. Figure 9 shows the processing images in the segmented hand area. The dataset images were collected by a webcam and had a pixel resolution of 128×128 . Three hundred images were captured for each gesture function. A total of 12,600 images of seven classes, including 10,080 training images and 2520 test images, were used to extract the features, and these features were provided to the classifier for classification. The gestures were performed by the subject with the right hand only. Each hand gesture involved motion of the hand and/or fingers.

4.2 Experimental results

4.2.1 Hand gesture recognition

In this subsection, we evaluate the different hand gestures in two ways: i) offline performance; and ii) real-time user performance. In the offline evaluation, the proposed CNN architecture is trained on the entire dataset. We use a built-in dataset containing seven classes of six different subjects, with 12,600 hand gesture images. We used 80% of the images for training purposes,

Table 4 Results of hand gesture recognition

Gestures	Number of test gestures	Number of classified gestures	Number of misclassified gestures	Recognition accuracy (%)
Newline	360	354	6	98.33
Space	360	352	8	97.78
Delete	360	348	12	96.67
Backspace	360	345	15	95.83
Open	360	360	0	100
Close	360	360	0	100
Select Language	360	353	7	98.06
Sum and Rates	2520	2472	48	98.09

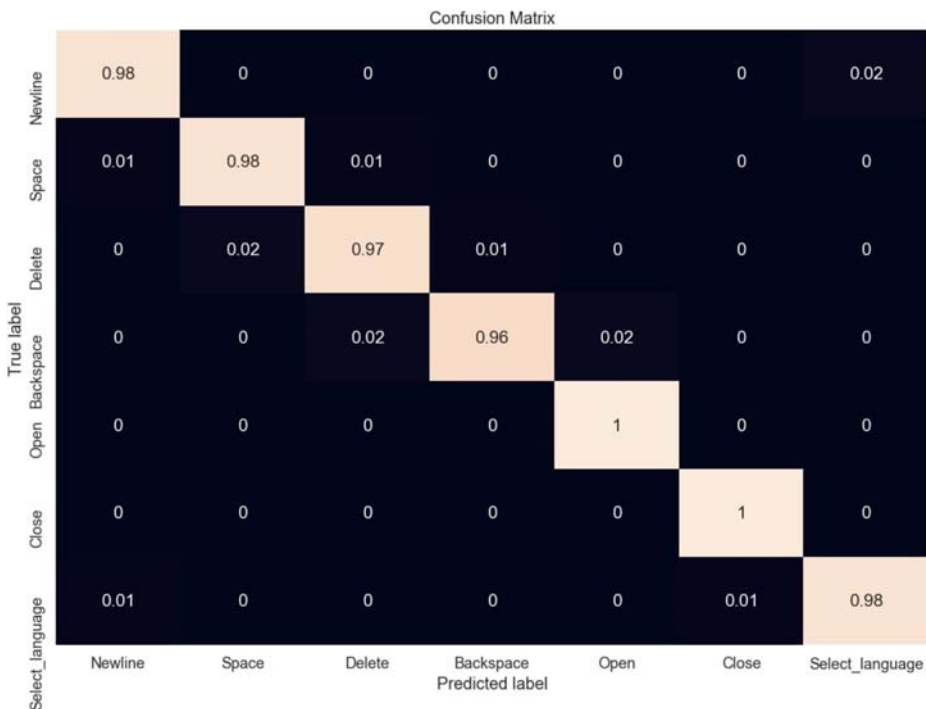


Fig. 12 Confusion matrix of hand gesture recognition accuracy

and 20% for testing. A total of $10,080 \times 4096$ features were extracted for training and 2520×4096 for testing. These features help in the classification of different hand gesture functions. Figure 10 illustrates a method of hand gesture recognition that is based on a supervised learning technique. In this method, the input images of typical hand gestures are pre-processed and segmented. In the training process, the system is pre-trained based on the feature gestures obtained using CNN. The extracted features are used to train the SVM. In the recognition process, the features are extracted from the user's new gesture and classified by trained SVM. Pre-training process enhances model performance and recognition accuracy.

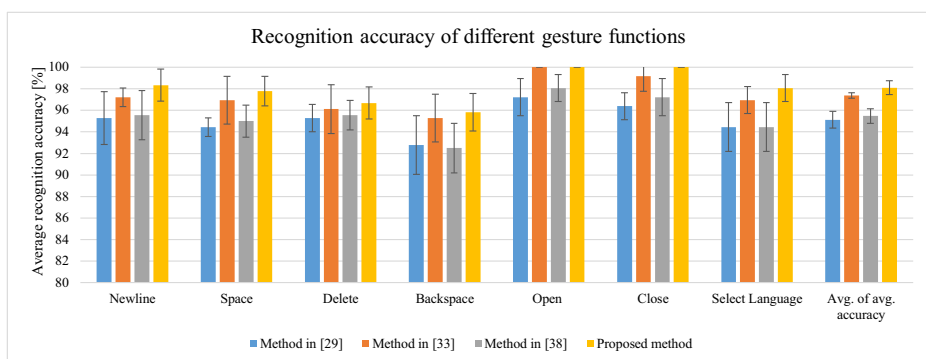


Fig. 13 Comparison the accuracy of recognizing different gesture functions of the proposed and state-of-art method

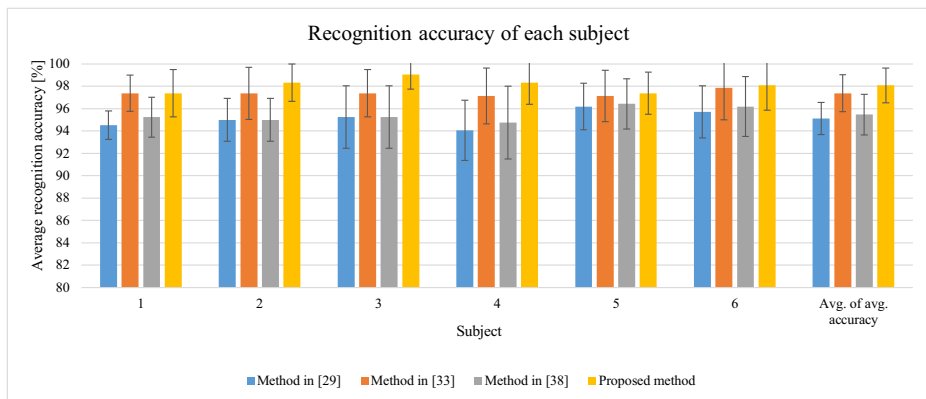


Fig. 14 Comparison the accuracy of recognizing different gesture functions for each user of the proposed and state-of-art method

In this experiment, we trained our model for 15 epochs. The accuracy and losses of our model are shown in Fig. 11a, b. The results are given in Table 3, where the average accuracy of the correct gesture recognition is about 98.09%. The ‘open’ and ‘close’ gesture functions achieved the highest acceptance accuracy of 100%, while the ‘backspace’ gesture had the lowest accuracy at 95.83%. Table 4 shows the results of classified and misclassified hand gesture. The confusion matrix of hand gesture recognition accuracy is shown in Fig. 12. As a comparison, we applied the method of CNN and de-noising [29], double channel CNN [33] and the skin model and the calibration of hand position [38] to obtain training and testing data. The average recognition accuracy of different gesture functions was obtained 95.12%, 97.38% and 95.48%, respectively. Figure 13 shows the average recognition accuracy of different gesture functions of proposed method and state-of-art method. The comparison of recognition accuracy of different gesture functions for each subject is shown in Fig. 14. A comparison of hand gesture recognition using our dataset with state-of-the-arts algorithm is presented in Table 5. In comparison to these results we can say that the experimental evaluation of our proposed approach recognizes better accuracy. The real-time user’s performances of hand-gesture recognition are described in the following subsections.

4.2.2 Non-touch character input

In this subsection, we evaluate the accuracy of character selection and the speed of the character input. In this experiment, we specified a list of gestures for each respondent and asked them to input characters and different gesture functions in a certain order in the experimental environment. Each user input a total of 55 characters, including English, Japanese and numbers. The sentences were “あいづわかまつのあしたのてんきははれですRoom

Table 5 Comparison of recognition accuracy with state-of-the-art algorithms (Evaluated using our dataset)

Reference	Method	Reported accuracy (%)	Classification accuracy (%)
[29]	CNN and stacked de-noising	92.83	95.12
[33]	Double channel CNN	98.02	97.38
[38]	Skin model and CNN	95.96	95.48
Proposed	HSV segmentation + CNN + SVM	—	98.09

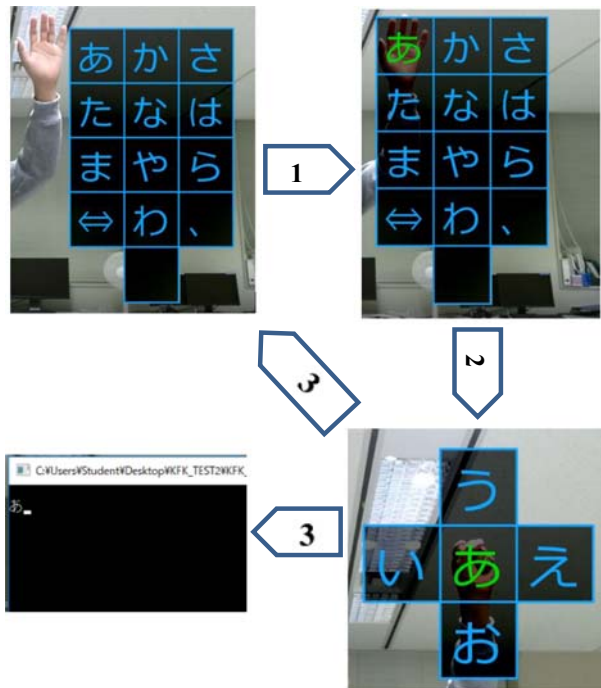


Fig. 15 Procedure used in the gestural flick input system

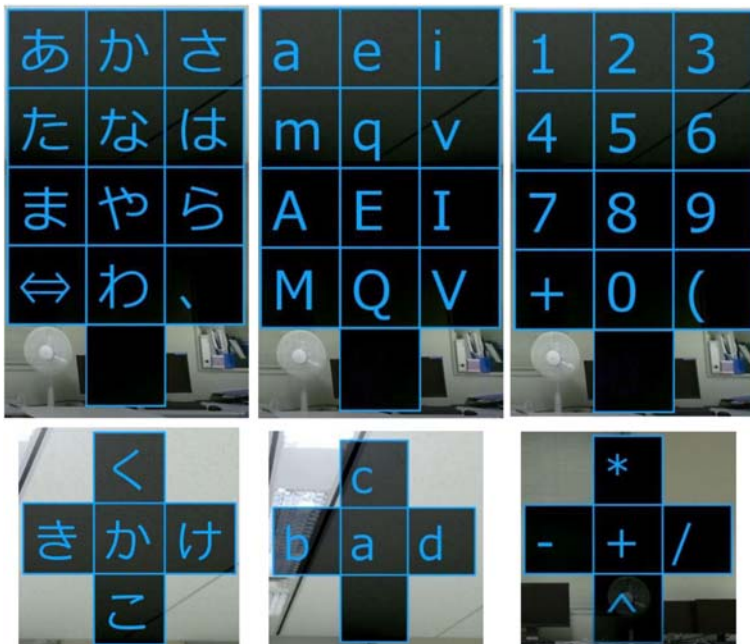


Fig. 16 Example of PVK and SVK display screens



Fig. 17 Character input simulation using the virtual keyboard

no: 301 Pattern Processing Lab”. Twenty-five volunteers participated in these experiments. The volunteers chose characters using the virtual keyboard and entered the appropriate characters as input. The gestural flick input system is shown in Fig. 15. Inputting the desired characters, the user performed the ‘close’ gesture via the first character of the flick character sequences on the PVK. The system then displayed the secondary virtual keyboard. The user then selected the appropriate character for input by performing the ‘close’ gesture. The selected characters were input, and the system again showed the primary virtual keyboard. Figure 16

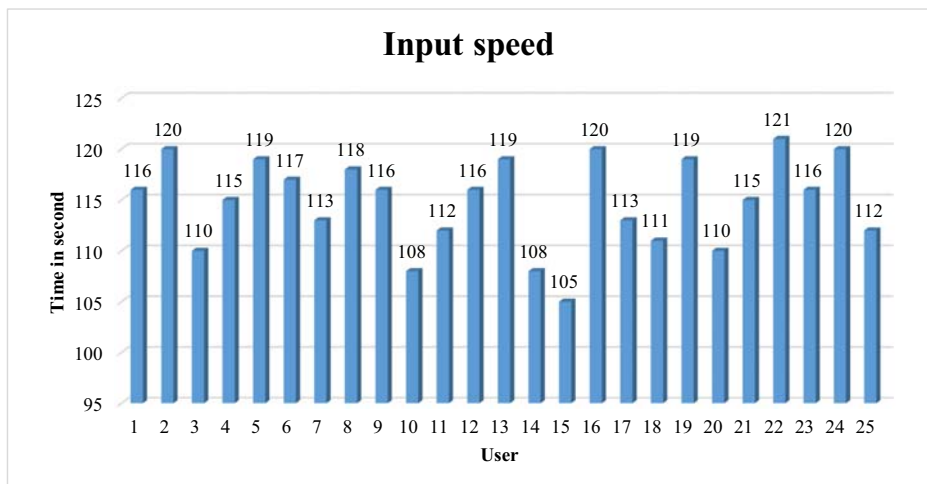


Fig. 18 Acquisition speed of the entire experiment for each user

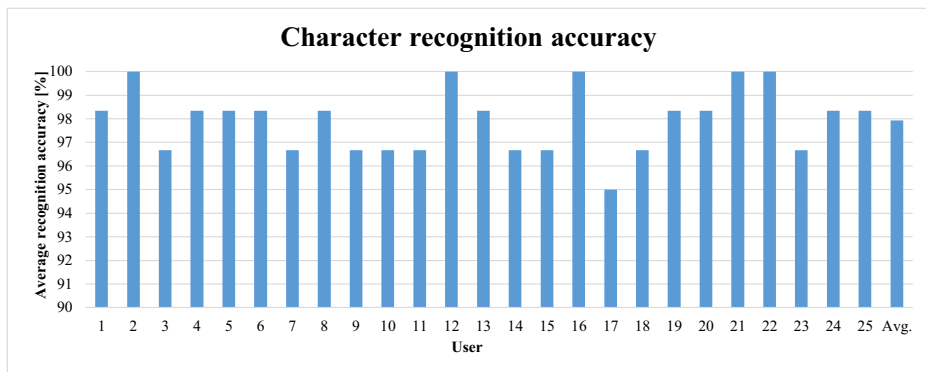


Fig. 19 Average character recognition accuracy of all users

shows the virtual keyboard screen displays of different character sets. To capture the gesture, the system uses a maximum position of one second; if the system does not receive any gesture activity within the ROI within one second, the gesture is not accepted. If the user fails to perform the gesture within the suggested time, the system cancels the gesture at this time and waits for the next gesture. The ‘open’ gesture acts as a no gesture activity, however, is used for the hand movement for character selection and input performance. Figure 17 depicts an example of the character input system. The character input speed and recognition accuracy of the entire experiment for each user are shown in Figs. 18 and 19, respectively, and the average recognition accuracy of character input and the speed of input are given in Table 6. The character recognition rate of this system was 100%, although there were some errors in selecting the character based on gesture functions in this system. One reason for incorrect character recognition was that the user performed the gesture functions near to the character borders. Despite this, the proposed system achieved a 2.07% error rate for character selection, as shown in Table 5. The overall accuracy of character recognition was 97.93%. Moreover, the average speed of the character input was 29.0 [characters/min], and the speed of character input using our system is therefore satisfactory. The time performances of the character inputs compared with the state-of-the-art systems are presented in Table 7. The recognition accuracy compared to some previous studies as accuracy of character input and different gesture function recognition are given in Table 8.

During the real-time user performances, we evaluated the recognition accuracy of different gesture functions for different users. The users were asked to perform each gesture five times. The system evaluated each gesture during processing, which involved image processing, feature extraction and gesture recognition. The total time required was 28.03 ms which is sufficient for real-time recognition at 30fps. However, the frame rate was limited by the

Table 6 Average recognition accuracy of character input and input performance

No. of users	No. of characters	Input speed [characters/min]	Avg. input Speed [in s]	Avg. character recognition accuracy (%)
25	55	29.0	114	97.93

Table 7 Comparison of character input time with state-of-the-art systems

Reference	Functions	Average time per character	Average error rate (%)
[18]	Air Writing recognition	2.55 s	1.9%
[19]	Virtual keyboard writing	1.46 s	9.84%
[21]	Eye-writing	15 s	12.75%
[22]	Handwriting recognition	0.83 s	7%
Proposed	Character input	2.08 s	2.07%

webcam to 33.33 ms per frame. Figure 20 shows the average recognition accuracies for the different gestures, and Fig. 21 presents the average recognition accuracy for different users and different gestures. From these results, we can see that the ‘open’ and ‘close’ gestures achieved higher recognition accuracy than the other gestures. However, the average recognition rate of the gesture functions in real time for this technique was 96.23%.

5 Discussion

In this study, we endeavor to recognize the character writing with seven typical hand gestures. The proposed system uses a virtual keyboard to produce characters, and the user input the character by performing the gesture at the ROI position on this keyboard. Based on the experimental results, the average accuracy of typical hand gestures is 98.09% as shown in Table 3. The recognition accuracy of each gesture was well-balanced to demonstrate the high reliability of the system. From the confusion matrix in Fig. 12, we identified some of the misrecognition of gesture recognition. The average misclassified gestures are 1.91% shown in Table 4. This type of error was caused by the gesture shape and size of the hand. Apparently distinguishable gestures like “backspace” and “open” can look the same when performing hand gestures. However, the ‘open’ and ‘close’ achieves highest accuracy where the ‘backspace’ is the lowest. Further, we evaluated various state-of-the-art algorithms using our dataset to compare the accuracy of hand gesture recognition shown in Table 5 and achieved 95.12%, 97.38% and 95.48%, respectively. We observed that the method [33] achieved the closest 97.38% to the achievement of our proposed method. This method used input images and segmented images for CNN’s double channels. However, the noise and segmentation process of the input images can have an impact on the accuracy. From the results, we can say that our

Table 8 Recognition accuracy comparison with other related studies

Functions	Reference	Recognition accuracy
Hand tapping gesture detection	[15]	Character input (94.3%)
Uni-stroke character input	[17]	Character input (96.5%)
Air pen system	[23]	Gesture signal (96.7%)
Hand gesture based writing	[27]	Air-writing (96.9%)
Hand gesture recognition	[38]	Gesture recognition (95.96%)
Fingertip detection	[39]	Character input (66%)
Hand gesture recognition	Proposed	Character input (97.93%)
		Gesture recognition (98.09%)

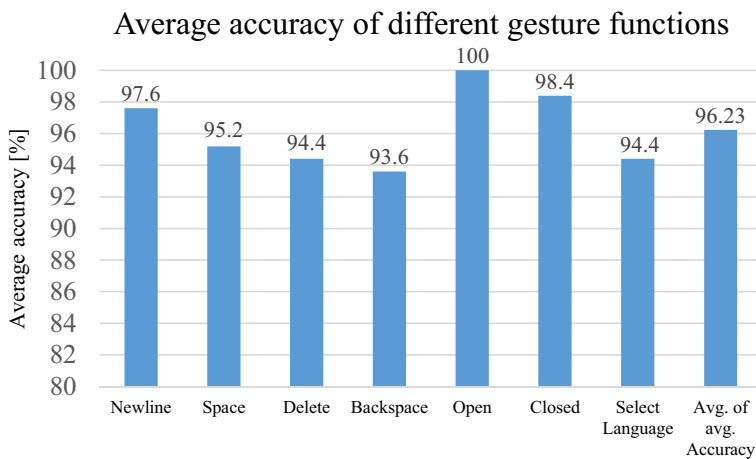


Fig. 20 Average recognition accuracy of different gesture functions

proposed method achieves highest accuracy compared to state-of-the-art algorithms. Also, real-time evaluations were conducted on different gestures activities among different users whose accuracy was recognized as 96.23% shown in Fig. 20.

Based on the obtained results, the recognition accuracy of character input is 97.93% shown in Table 6. The results indicate that the performance of the proposed system was robust among users although different users showed variability in writing speed. The average speed of writing characters is 114 s shown in Table 6. An example of character writing speed among the users is illustrated in Fig. 18. Despite the inherent variability among users, the non-touch character writing system achieves a maximum accuracy of 100% and a minimum of 95% is shown in Fig. 19. The reason for the minimum accuracy is that the user selects the character near the character boundary which produces the incorrect character. Moreover, we compared the time performances and the recognition accuracy of character input is shown in Tables 7 and 8, respectively. The evaluation results confirm that this system achieves higher recognition accuracy than conventional one. Compared to the character input, the user performs a character in 2.08 s with a 2.07% error rate is

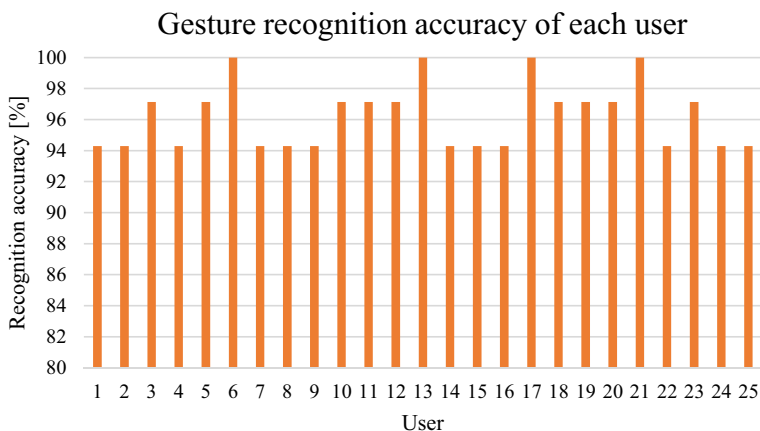


Fig. 21 Average accuracy of different gesture functions for each user

shown in Table 7. An example of [22], the average time per character is 0.83 s, nevertheless the error rate is 7%. In another [18] the error rate is 1.9% nevertheless the average time of the character is 2.55 s. The average time for each character is 1.46 s using a virtual keyboard [19], though the character error rate is higher. As a result, we can conclude that users can easily perform character inputs using this system in a relatively short time, with minimal error rates.

6 Conclusion

In this paper, we present a non-touch character writing system based on hand gestures that provide a reliable and efficient interface for character input for the most demanding environments and applications, reducing the gap between humans, machines or robots, and allowing the user to operate the system in a healthy and safe way. The proposed method can play an effective role in gesture recognition and character inputs. Hand gesture recognition is the main concern of character input. From this point of view, this paper proposed a hand gesture recognition model where gesture images were captured from the region of interest (ROI) of the virtual keyboard. The gesture image was segmented to detect the hand area and the proposed CNN method was used to extract the gesture features. Finally, the gestures were recognized using SVM classification techniques. This hand gesture recognition system was incorporated in the proposed non-touch character input system. The user generates character input by viewing a virtual keyboard on a monitor using different hand gesture functions. A flick keyboard system was introduced to the virtual keyboard, through which the user can quickly and easily input characters. We used seven gestures and applied a robustness color segmentation scheme for detecting the hand. In order to verify our proposed method, an experimental setup was established in a laboratory environment, in which 25 volunteers participated. The hand gesture images are collected using a low-cost device like a webcam. The experimental results show that our system can achieve a high recognition accuracy in offline and real-time conditions of 98.09% and 96.23%, respectively, with seven general gesture functions, and average recognition accuracy for character input of 97.93%. It is robust in a real-time environment. Also, recognition accuracy shows satisfactory performances compared to the state-of-art method.

In the future, we will focus on reducing error rates and increasing performance over time compared to state-of-the-art systems, and add more typical hand gestures for character inputs with different languages that support greater functionality and instruction for advanced non-touch systems.

References

1. Yin H, Zhou A, Liu L, Wang N, Ma H (2019) Ubiquitous writer: robust text input for small mobile devices via acoustic sensing. *IEEE Internet Things J* 6(3):5285–5296
2. Kumar P, Saini R, Behera SK, Dogra DP, Roy PP (2017) Real-time recognition of sign language gestures and air-writing using leap motion. In: *Machine vision applications (MVA), IEEE 15th IAPR international conference on*, pp. 157–160
3. Ben Jmaa A, Mahdi W, Ben Jemaa Y, Ben Hamadou A (2016) A new approach for hand gestures recognition based on depth map captured by RGB-D camera. *Computación y Sistemas* 20(4):709–721
4. Machatake M, Nishida M, Horiuchi Y, Ichikawa A (2007) A study on sign language recognition based on gesture components of position and movement (in Japanese). In: *Proc. Workshop Interact. Syst. Softw. (WISS), Japan*, pp. 129–130

5. Nishimura Y, Inamura D, Horiuchi Y, Kawamoto K, Shinozaki T (2012) HMM sign language recognition using Kinect and particle filter. *Inst Electron Inf Commun Eng Pattern Recogn Media Underst* 111(430): 161–166
6. Sagayam KM, Hemanth DJ (2017) Hand posture and gesture recognition techniques for virtual reality applications: a survey. *Virtual Reality* 21(2):91–107
7. Mahmoudi MT, Mojtahedi S, Shams S (2017) AR-based value-added visualization of infographic for enhancing learning performance. *Comput Appl Eng Educ* 25(6):1038–1052
8. Rahim M A, Shin J, Islam M R (2018) Human-machine interaction based on hand gesture recognition using skeleton information of Kinect sensor. In: *Proceedings of the 3rd international conference on applications in information technology*, pp. 75–79, ACM
9. Rautaray SS, Agrawal A (2015) Vision based hand gesture recognition for human computer interaction: a survey. *Artif Intell Rev* 43(1):1–54
10. Ohn-Bar E, Trivedi MM (2014) Hand gesture recognition in real time for automotive interfaces: a multimodal vision-based approach and evaluations. *IEEE Trans Intell Transp Syst* 15(6):2368–2377
11. Microsoft X-box Kinect, <http://xbox.com>
12. Leap motion, <http://www.leapmotion.com>
13. Ishiyama H, Kurabayashi S (2016) Monochrome glove: a robust real-time hand gesture recognition method by using a fabric glove with design of structured markers. In: *Virtual reality (VR)*, pp. 187–188, IEEE
14. Ho N, Wong PM, Chua M, Chui CK (2018) Virtual reality training for assembly of hybrid medical devices. *Multimed Tools Appl* 77(23):30651–30682
15. Shin J, Kim CM (2017) Non-touch character input system based on hand tapping gestures using Kinect sensor. *IEEE Access* 5:10496–10505
16. Mitra S, Acharya T (2007) Gesture recognition: a survey. *IEEE Trans Syst Man Cybern Part C Appl Rev* 37(3):311–324
17. Kane L, Khanna P (2017) Vision-based mid-air unistroke character input using polar signatures. *IEEE Trans Hum Mach Syst* 47(6):1077–1088
18. Chen M, AlRegib G, Juang BH (2016) Air-writing recognition—part I: modeling and recognition of characters, words, and connecting motions. *IEEE Trans Hum Mach Syst* 46(3):403–413
19. Chen M, AlRegib G, Juang BH (2016) Air-writing recognition—part II: detection and recognition of writing activity in continuous stream of motion data. *IEEE Trans Hum Mach Syst* 46(3):436–444
20. Wang RY, Popović J (2009) Real-time hand-tracking with a color glove. *ACM Trans Graph* 28(3):63
21. Lee KR, Chang WD, Kim S, Im CH (2016) Real-time “eye-writing” recognition using electrooculogram. *IEEE Trans Neural Syst Rehabil Eng* 25(1):37–48
22. Amma C, Georgi M, Schultz T (2014) Airwriting: a wearable handwriting recognition system. *Pers Ubiquit Comput* 18(1):191–203
23. Li Q, Cao H, Lu Y, Yan H, Li T (2016) Controlling non-touch screens as touch screens using Airpen, a writing tool with in-air gesturing mode. In: *System and software reliability (ISSSR), international symposium on*, pp. 68–76. IEEE
24. Chua C, Guan H, Ho Y (2002) Model-based 3D hand posture estimation from a single 2D image. *Image Vis Comput* 20:191–202
25. Lowe D (1991) Fitting parameterized 3D models to images. *IEEE Trans Pattern Anal Mach Intell* 13:441–450
26. Ren Z, Yuan J, Meng J, Zhang Z (2016) Robust part-based hand gesture recognition using Kinect sensor. *IEEE Trans Multimedia* 15(5):1110–1120
27. Murata T, Shin J (2014) Hand gesture and character recognition based on Kinect sensor. *Int J Distrib Sens Netw* 10(7):278460
28. Neverova N, Wolf C, Taylor GW, Nebout F (2014) Multi-scale deep learning for gesture detection and localization. In: *Computer vision-ECCV 2014 workshops*. Springer, Heidelberg, pp 474–490
29. Oyedotun OK, Khashman A (2017) Deep learning in vision-based static hand gesture recognition. *Neural Comput & Applic* 28(12):3941–3951
30. Wang P, Li W, Gao Z, Tang C, Zhang J, Ogunbona P O (2015) Convnets-based action recognition from depth maps through virtual cameras and pseudocoloring. In: *Proc. ACM Int. Conf. Multimedia*, pp. 1119–1122
31. Wang P, Li W, Gao Z, Zhang J, Tang C, Ogunbona PO (2016) Action recognition from depth maps using deep convolutional neural networks. *IEEE Trans Human-Mach Syst* 46(4):498–509
32. Hou Y, Li Z, Wang P, Li W (2018) Skeleton optical spectra-based action recognition using convolutional neural networks. *IEEE Trans Circuits Syst Video Technol* 28(3):807–811
33. Yan Wu X (2019) A hand gesture recognition algorithm based on DC-CNN. *Multimed Tools Appl* 1–13
34. Rautaray S S, Agrawal A (2015) Vision based hand gesture recognition for human computer interaction: A survey. *Artificial Intelligence Review* 43:1–54.
35. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. *Nature* 521(7553):436–444

36. Krizhevsky A, Sutskever I, Hinton G E (2012) Imagenet classification with deep convolutional neural networks. In Proceedings of the 25th international conference on neural information processing systems, pp. 1097–1105
37. Islam MR, Uddin J, Kim J-M (2016) Acoustic emission sensor network based fault diagnosis of induction motors using a Gabor filter and multiclass support vector machines. *Adhoc Sens Wirel Netw* 34:273–287
38. Lin HI, Hsu MH, Chen WK (2014) Human hand gesture recognition using a convolution neural network. In IEEE international conference on automation science and engineering (CASE), pp. 1038–1043
39. Matsui N, Yamamoto Y (2000) Virtual keyboard: realization of robust real-time fingertip detection from video-image. (in Japanese), in Proc. Japan Soc. Softw. Sci. Technol. (JSSST), pp. 1–7

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Md Abdur Rahim is currently working toward the Ph.D. degrees in the Pattern Processing Laboratory, Graduate School of Computer Science and Engineering, The University of Aizu, Fukushima, Japan. He is working (now on study leave) as an assistant professor, Department of Computer Science and Engineering, Pabna University of Science and Technology, Bangladesh. He received the Bachelor of Science (Honours) and Master of Science (M.Sc.) degrees in Computer Science and Engineering from University of Rajshahi, Bangladesh in 2008 and 2009. His research interests include human computer interaction, pattern recognition, computer vision and image processing, human recognition and machine intelligence.



Jungpil Shin received a Ph.D. in Communication Engineering from Kyushu University, Japan in 1999. He became an Associate Professor and Senior Associate Professor in the School of Computer Science and

Engineering, the University of Aizu, Japan, in 1999 and 2004, respectively. His research interests include pattern recognition, character recognition, image processing, and computer vision. He is currently researching the following advanced fields: pen-based interaction system, real-time system, oriental character processing, mobile computing, computer education, human recognition, and machine intelligence. He served several conferences as a program chair and program committee member for numerous international conferences and serves as a reviewer for several SCI major journals and an editor of journals. He is a member of IEEE, ACM, IEICE, IPSJ, KISS, and KIPS.



Md. Rashedul Islam received the B.S. degree in computer science and engineering from the University Rajshahi, Rajshahi, Bangladesh, in 2006, the M.S. degree in informatics from the Höskolan i Borås (University of Borås), Borås, Sweden, in 2011, and Ph.D. degree in electrical, electronic, and computer engineering at the University of Ulsan, Ulsan, South Korea, in 2016. He is currently working as a visiting researcher in University of Aizu, Japan also an Associate Professor (on study leave) in the Department of Computer Science and Engineering, University of Asia Pacific (UAP), Dhaka, Bangladesh. His research interests include Signal & Image processing, Machine learning, Feature selection and optimization, Human computer interaction, Data-driven bearing fault diagnosis and prognostics, and parallel algorithm/processing. He has several publications in major journals (SCI, SCIE) and conferences. He is reviewer of few SCIE journals. He has contributed to several conferences as organizing secretary, member of program committee, session chair and a reviewer.

Affiliations

Md. Abdur Rahim¹ • Jungpil Shin¹ • Md. Rashedul Islam¹

Md. Abdur Rahim
rahim_bds@yahoo.com

Md. Rashedul Islam
rashed.cse@gmail.com

¹ School of Computer Science and Engineering, The University of Aizu, Aizu-wakamatsu, Fukushima 965-8580, Japan